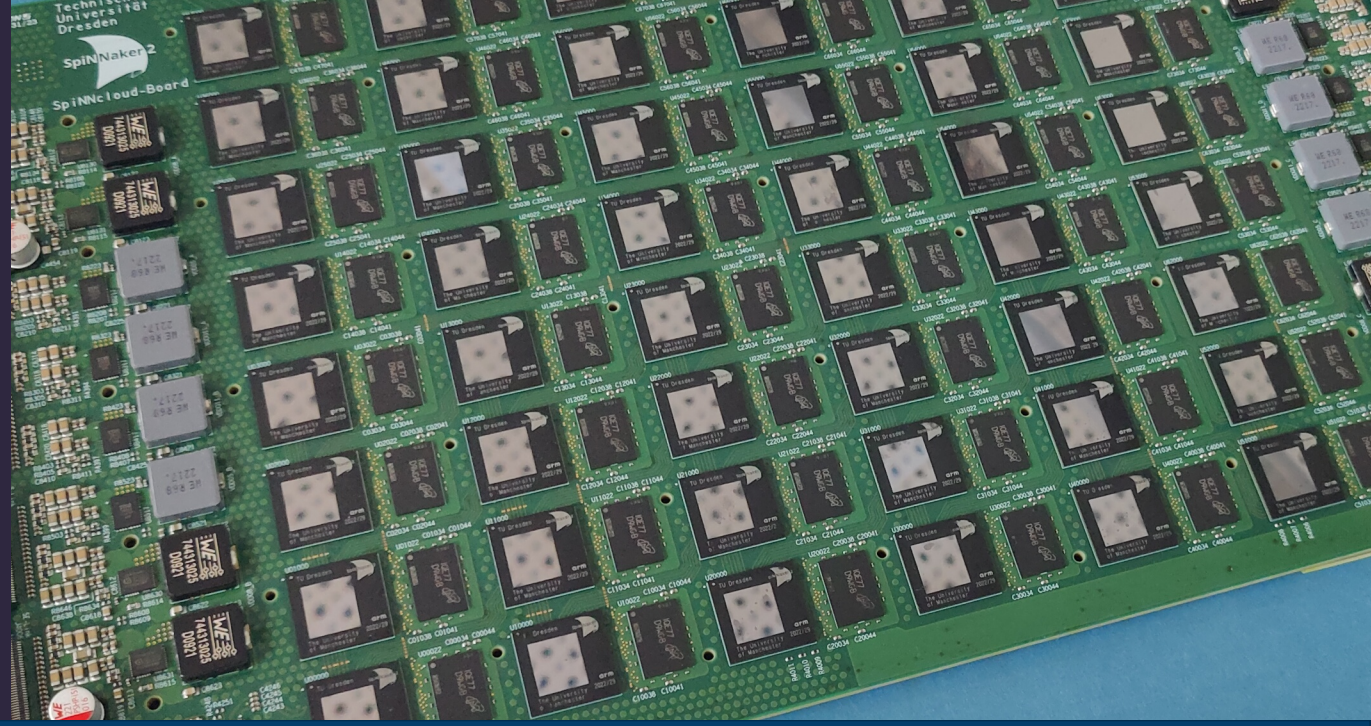
```
equations:
    kernel I_kernel = (e / tau_syn) * t * exp(-t / tau_syn)
    inline I_syn pA = convolve(I_kernel, spikes) * pA
    V_m' = -(V_m - E_L) / tau_m + (I_syn + I_e + I_stim) / C_m

parameters:
    C_m      pF = 250 pF      # Capacitance of the membrane
    tau_m    ms = 10 ms       # Membrane time constant
    tau_syn  ms = 2 ms        # Time constant of synaptic current
    refr_T   ms = 2 ms        # Duration of refractory period
    E_L      mV = -70 mV      # Resting potential
    V_reset  mV = -70 mV      # Reset potential of the membrane
    V_th     mV = -55 mV      # Spike threshold potential
    I_e      pA = 0 pA        # Constant external input current

input:
    spikes <- spike
    I_stim pA <- continuous

output:
    spike

update:
```



MODELING AND SIMULATING NEURON MODELS FROM NESTML ON NEUROMORPHIC HARDWARE

Charl Linssen <c.linssen@fz-juelich.de> | NEST Conference 2026 | June 16th, 2026

Human Brain Project
EBRAINS
Co-funded by the European Union
JÜLICH Forschungszentrum
Member of the Helmholtz Association

Welcome!

T+00:00	Tutorial overview and accounts setup
T+00:10	Introduction to NESTML
T+00:20	Hands-on part 1: Generating code
T+00:30	Hands-on part 2: Single-neuron characterisation
T+00:40	Hands-on part 3: Balanced network

JÜLICH
Forschungszentrum

Prerequisites: access SpiNNaker via EBRAINS

EBRAINS is an integrated digital brain research and simulation platform. It provides cloud computing services for free.

To run the software used in this tutorial on EBRAINS, please create an account at:

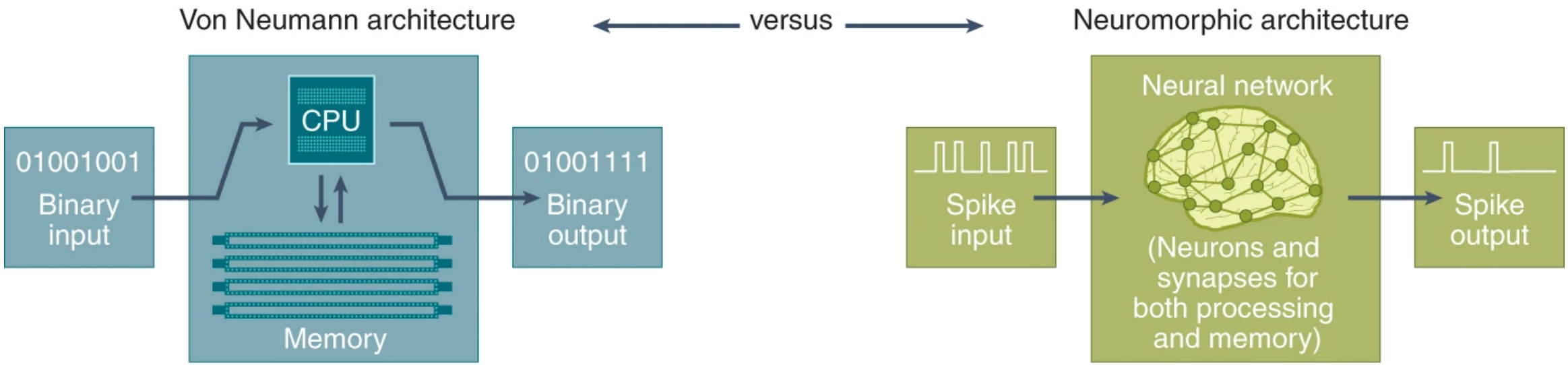
https://ebrains.eu/sign-up

EBRAINS

FET FLAGSHIPS
Co-funded by the European Union

JÜLICH Forschungszentrum

Introduction to neuromorphic computing



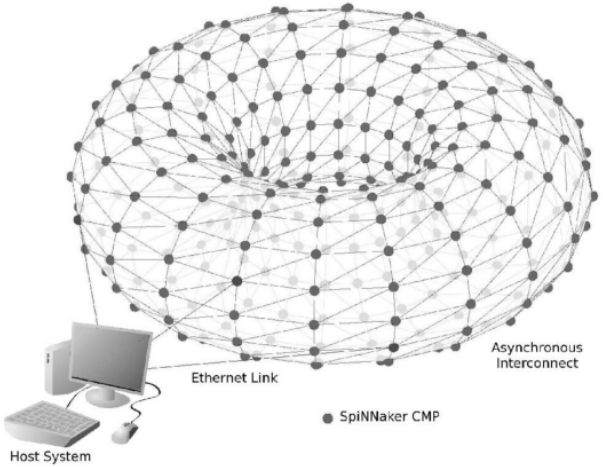
Von Neumann architecture		Operation		Neuromorphic architecture
Sequential processing	←	Operation	→	Massively parallel processing
Separated computation and memory	←	Organization	→	Collocated processing and memory
Code as binary instructions	←	Programming	→	Spiking neural network
Binary data	←	Communication	→	Spikes
Synchronous (clock-driven)	←	Timing	→	Asynchronous (event-driven)

Schuman et al. 2022

Introduction to SpiNNaker

SpiNNaker is a neuromorphic computing system designed to simulate large-scale spiking neural networks in real-time.

It features a massively parallel architecture comprising up to a million 32-bit ARM9 cores, interconnected through a toroidally wired, packet-based communication network, achieving high-speed, low-latency interprocessor communication.



JÜLICH
Forschungszentrum

Accessing SpiNNaker via EBRAINS

EBRAINS is an integrated digital brain research and simulation platform. It provides cloud computing services for free.

To run the software used in this tutorial on EBRAINS, please create an account at:

https://ebrains.eu/sign-up

EBRAINS

FET FLAGSHIPS

Co-funded by the European Union

JÜLICH Forschungszentrum

Python programming and JupyterLab

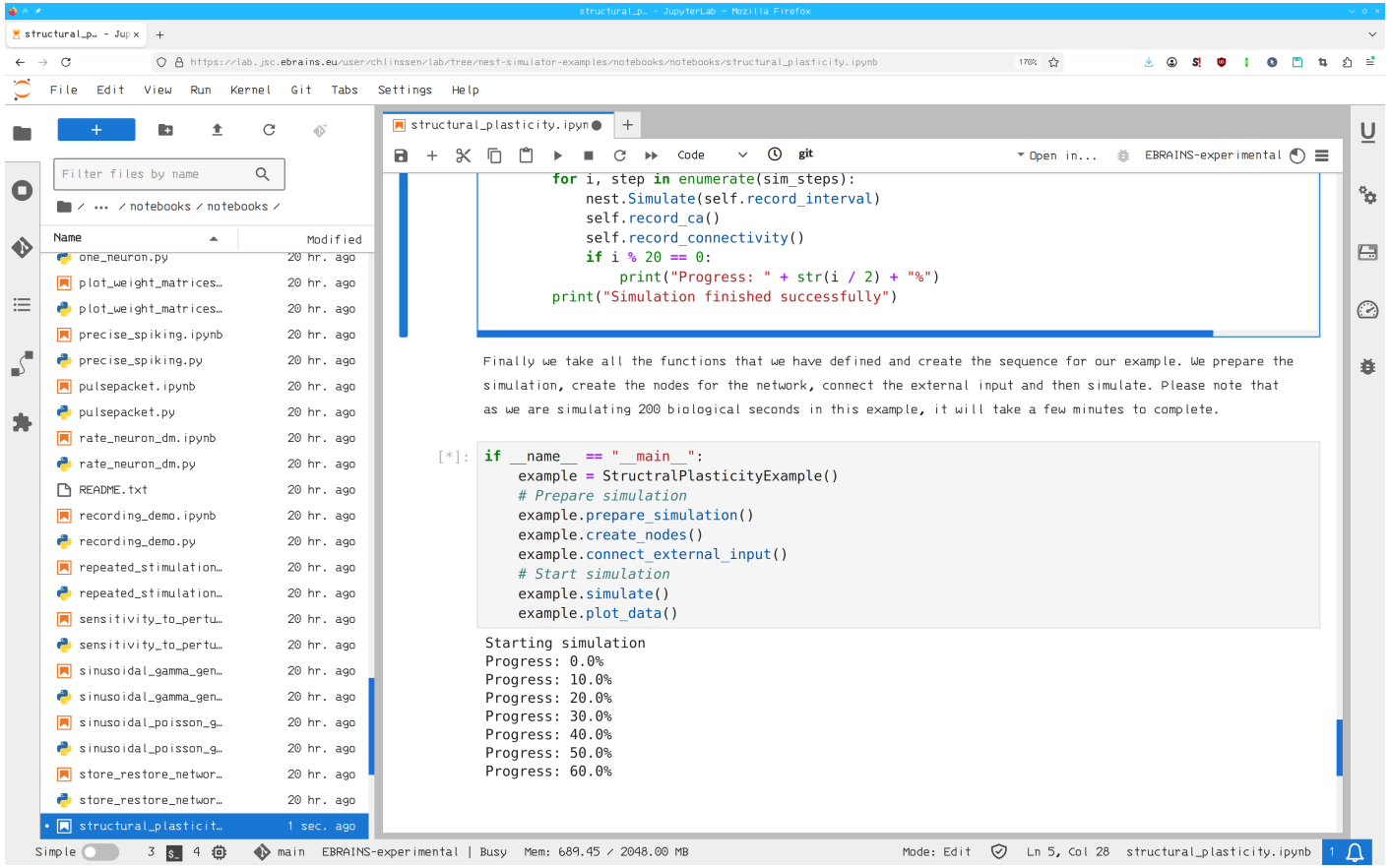
JupyterLab is an interactive graphical user interface (GUI) for the Python programming language.

Usage tips:

- Shift+Return runs a single cell.
- Use the ⏩ icon in the toolbar to run all cells.

For an advanced JupyerLab tutorial, see e.g. https://realpython.com/using-jupyterlab/

Code will run inside an EBRAINS VM with 2 GB RAM. Files stored in your VM home directory can disappear—save them locally for safekeeping!



JÜLICH Forschungszentrum

Where to find materials?

① Use your browser to go to the URL:

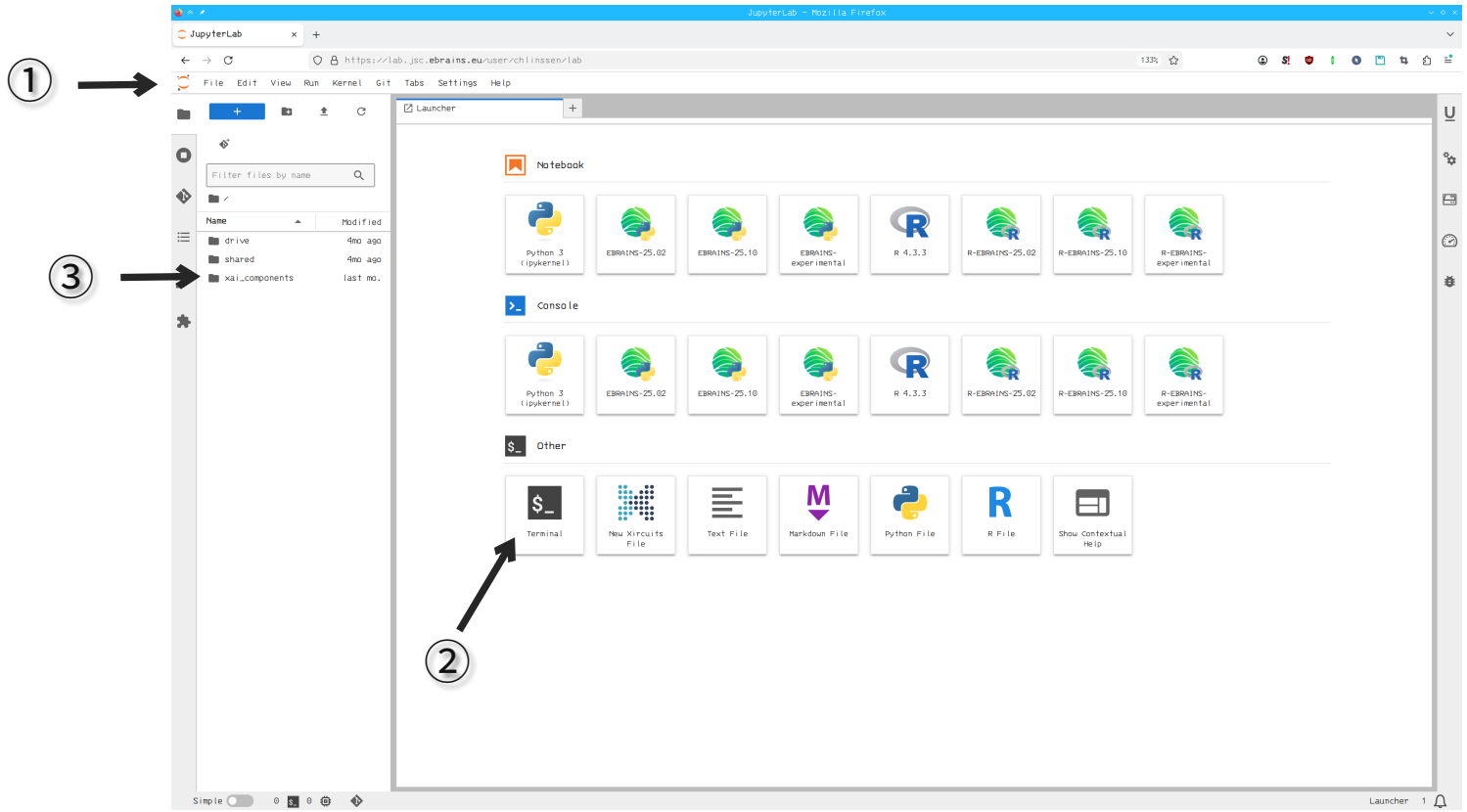
https://lab.ebrains.eu/

Select JSC as execution site.

② Open a terminal and run (one line):

```
git clone
https://github.com/clinssen/ebrai
ns-nestml-spinnaker
```

③ Open any of the downloaded notebooks



JÜLICH Forschungszentrum

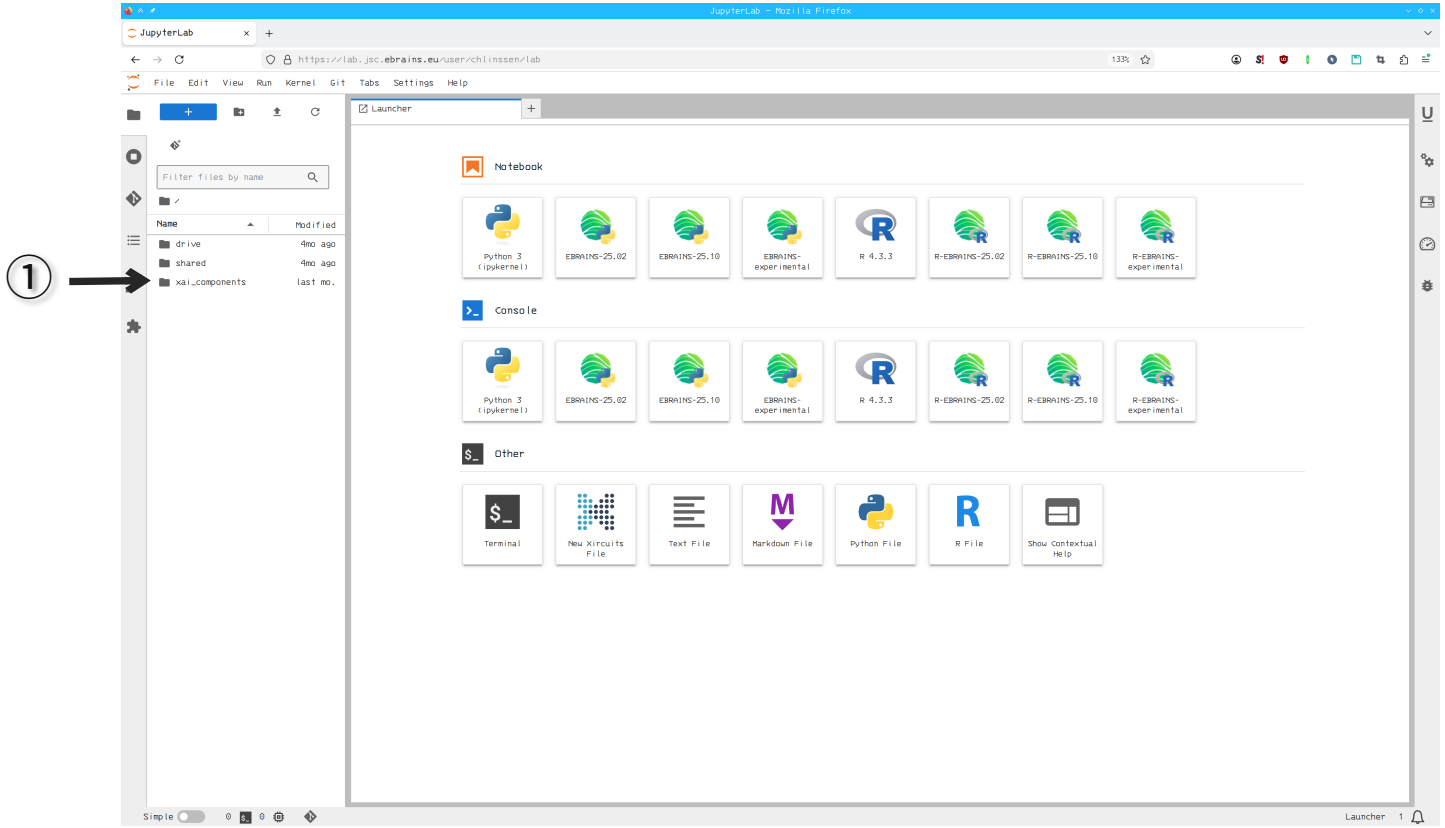
Part 1: Generating code for the neuron model

① Open the notebook:

`spinnaker_nestml_code_gen.ipynb`

② Run the entire notebook by clicking the ⏩ icon.

This may take a few minutes as it installs and compiles the entire SpiNNaker development software stack.



JÜLICH Forschungszentrum

Part 2: Characterising the single-neuron model

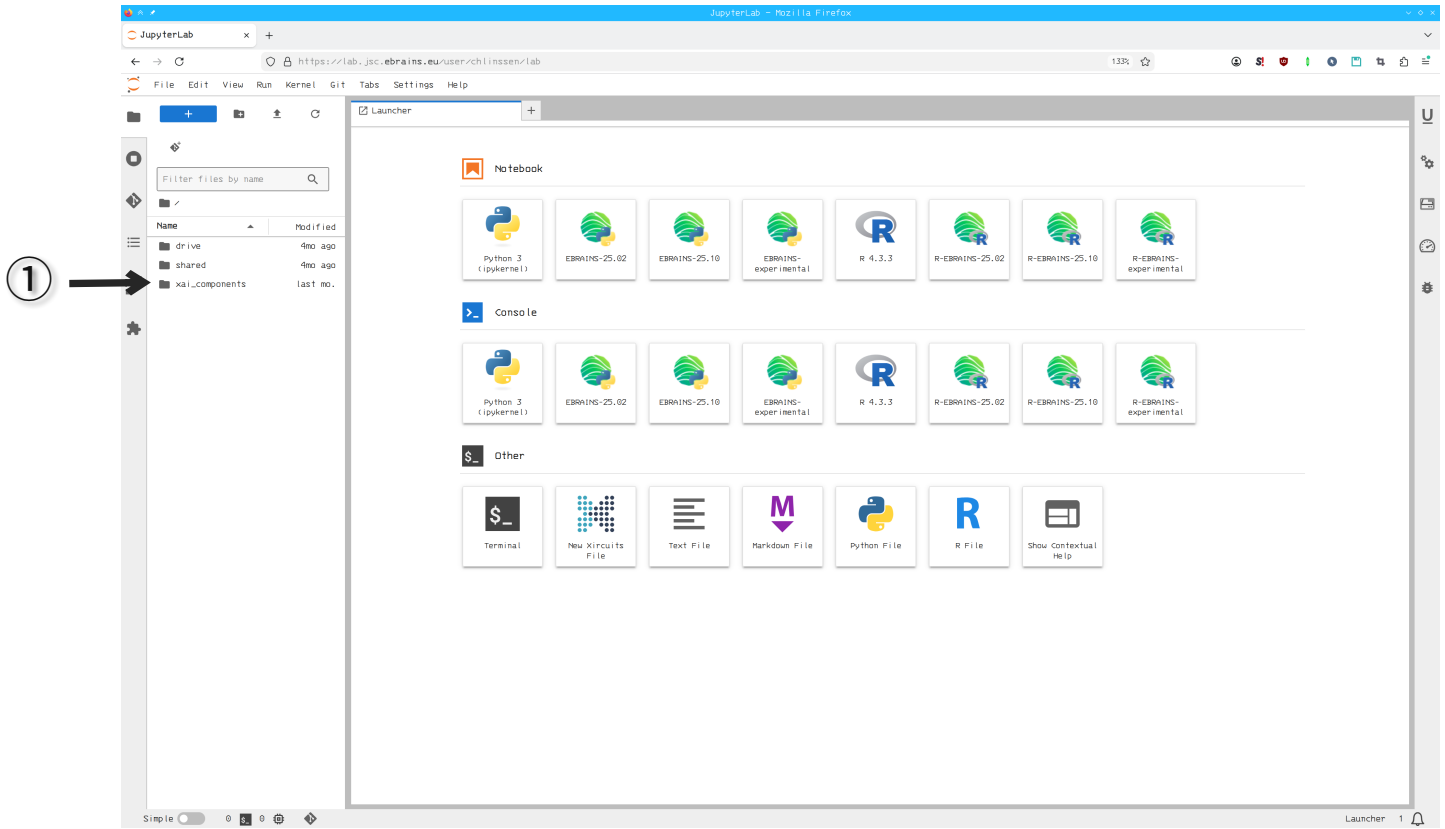
① Open the notebook:

`spinnaker_nestml_neuron_characterisation.ipynb`

② Run the entire notebook by clicking the ▶▶ icon.

A simulation will run that produces a plot of excitatory postsynaptic potentials.

③ Excercise: Add another spike generator for inhibitory spikes.



JÜLICH Forschungszentrum

Part 3: Balanced network model

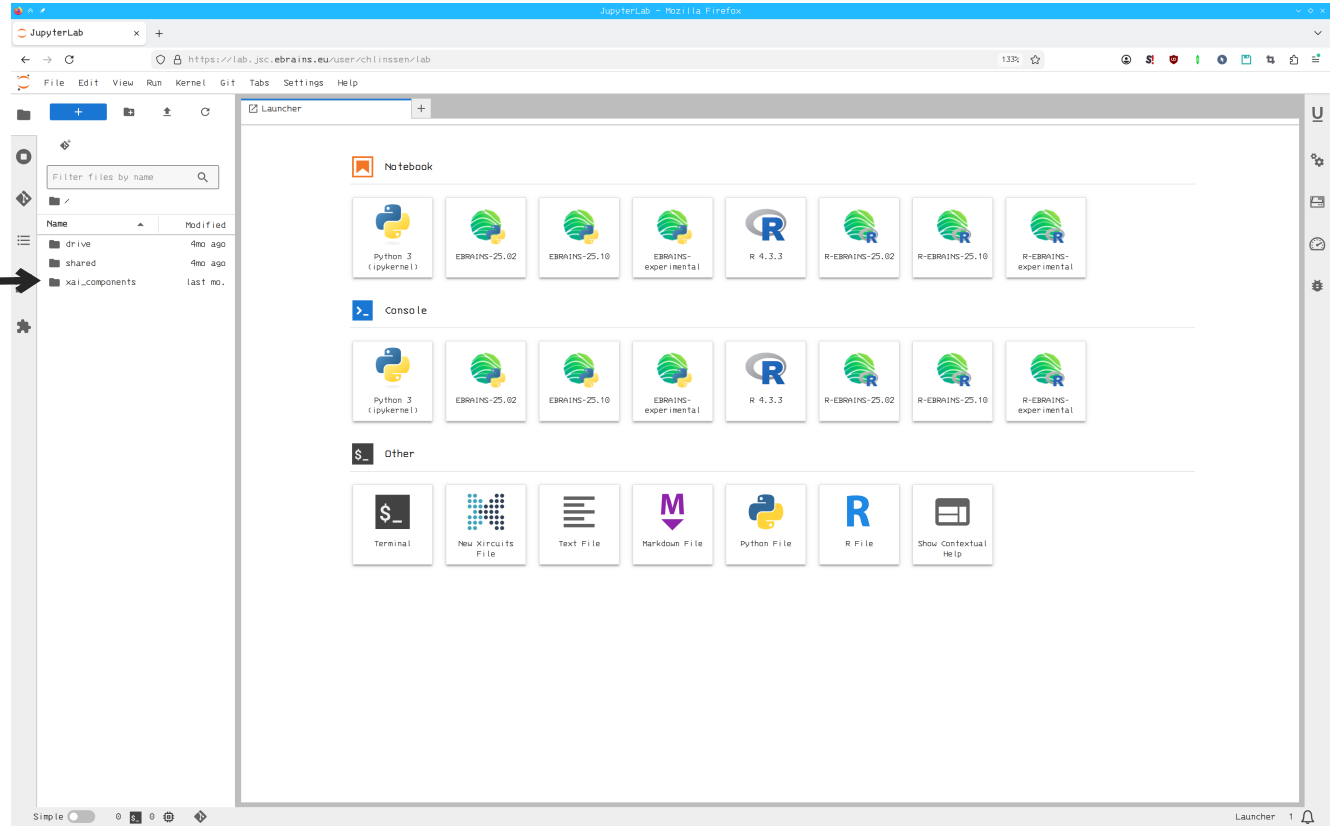
① Open the notebook:

`spinnaker_balanced_network.ipynb`

② Run the entire notebook by clicking the ▶▶ icon.

A simulation will run that produces a rastergram and a plot of membrane potentials.

③ Excercise: Change the ratio of excitation to inhibition and external input rate to achieve synchronous network firing.



JÜLICH Forschungszentrum

Further reading

NESTML:

https://nestml.readthedocs.io/

SpiNNaker:

https://spinnakermanchester.github.io/